

Stored XSS in Microweber v2.0.20

1. Vulnerability Description

A stored Cross-Site Scripting (XSS) vulnerability exists in Microweber v2.0.20.

The `/admin/users/<user-id>/edit` page does not properly sanitize or validate the **First Name** and **Last Name** input fields. A user who has permission to modify another user's profile can inject a crafted payload into these fields and trigger an XSS attack.

2. Affected Version

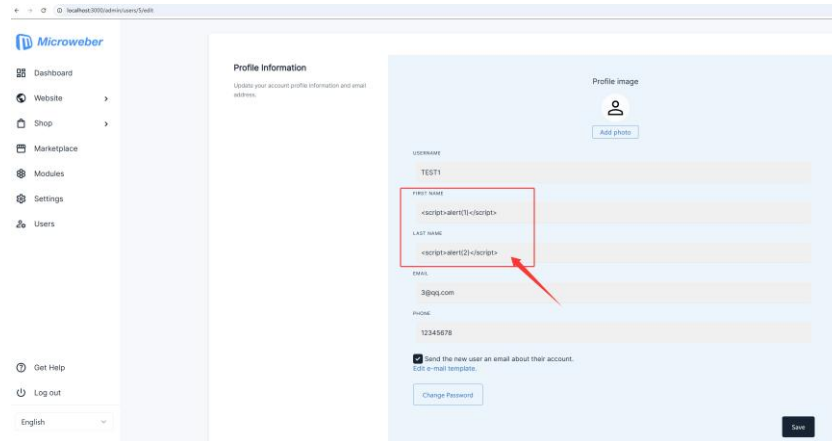
Microweber v2.0.20

Other versions have not been verified yet.

3. Steps to Reproduce

1. Log in as an administrator or any user who has permission to edit another user's profile.
2. Visit `http://localhost:3000/admin/users/<user-id>/edit`, where `` is the ID of the target user.

3. Inject a malicious payload into the 'First Name' or 'Last Name' field, then click the 'Save' button.



4. Ask the target user to log in.

5. After the target user logs in and visits a page that displays their profile name, the XSS payload is executed.



4. Proof of Concept Payload

A common XSS payload is sufficient to trigger the issue:

```
```html
```

```
<script>alert(1)</script>
```

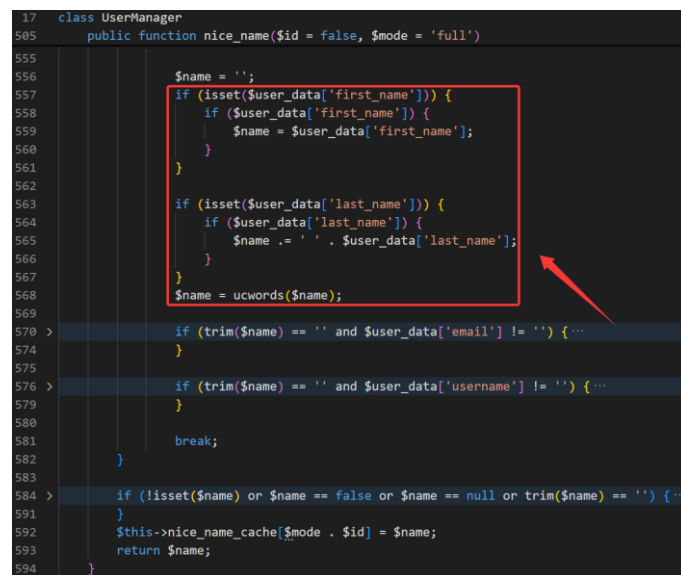
```
```
```

5. Vulnerable Code

1.\src\MicroweberPackages\User\UserManager.php#nice_name L505-L594

The nice_name() function is used to retrieve and build the user's display name from user-controlled fields such as first_name and last_name.

These values are concatenated and returned without proper escaping or sanitization.



```
17 class UserManager
505 public function nice_name($id = false, $mode = 'full')
555
556     $name = '';
557     if (isset($user_data['first_name'])) {
558         if ($user_data['first_name']) {
559             $name = $user_data['first_name'];
560         }
561     }
562
563     if (isset($user_data['last_name'])) {
564         if ($user_data['last_name']) {
565             $name .= ' ' . $user_data['last_name'];
566         }
567     }
568     $name = ucwords($name);
569
570 > if (trim($name) == '' and $user_data['email'] != '') { ...
574 }
575
576 > if (trim($name) == '' and $user_data['username'] != '') { ...
579 }
580
581     break;
582 }
583
584 > if (!isset($name) or $name == false or $name == null or trim($name) == '') { ...
591 }
592 $this->nice_name_cache[$mode . $id] = $name;
593 return $name;
594 }
```

2.\userfiles\modules\admin\dashboard.php#L4

Multiple pages output the returned user name directly without HTML escaping. dashboard.php is one example where the user name is rendered directly in the page, which leads to execution of the injected script.

```
userfiles > modules > admin > dashboard.php
1 <div class="col-xxl-8 col-xl-10 col-12 mx-auto">
2
3 <div class="dashboard-title-container ">
4 <h1 class="dashboard-welcome-title"><?php _e("Welcome back") ?>, <?php print user_name(); ?></h1>
5
6 <div class="d-flex flex-wrap justify-content-between align-items-center">
7 <div>
8 <p class="dashboard-welcome-p"><?php _e("Here's what's happening") ?></p>
9 </div>
10 <div>
11 <p class="dashboard-welcome-p"><?php _e("Last 30 days") ?></p>
12 </div>
13 </div>
14 </div>
```

6. Contact Information

You can contact me at: `sliao25@m.fudan.edu.cn`

If you determine that this report describes a valid security or privacy issue, I would appreciate being added to the related task or bug report so that I can follow its progress.